

Fiche de synthèse

Git au quotidien sur MiniForum

Développement web, classe passerelle

Usage toute l'année
Prérequis TP semaine 1

À quoi sert cette fiche

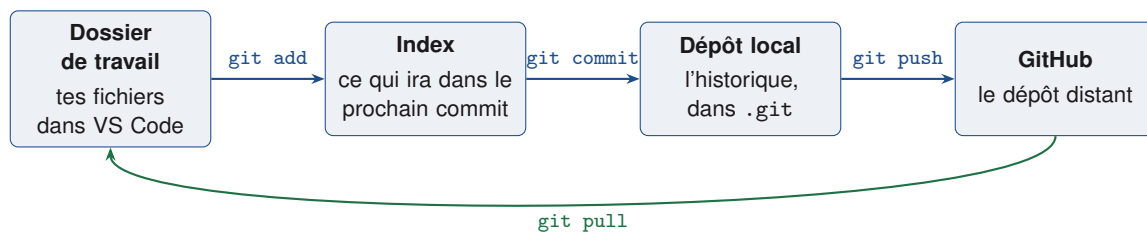
Le dépôt miniformum a été créé en semaine 1. Cette fiche décrit ce qui se fait **une seule fois**, ce qui se fait **à chaque changement**, et comment se sortir des situations qui bloquent le plus souvent. Garde-la ouverte pendant les TP : les commandes sont toujours les mêmes, tu les connaîtras par cœur d'ici la semaine 5.

Où taper ces commandes

Dans **Git Bash** (clic droit dans le dossier miniformum → *Open Git Bash here*) ou dans le terminal intégré de VS Code (*Terminal* → *Nouveau terminal*), à condition que le dossier ouvert soit bien miniformum.

1 Le principe en une image

Ton travail passe par quatre endroits. Chaque commande Git fait passer les fichiers de l'un à l'autre.



Trois erreurs de débutant que ce schéma évite

- Un `git commit` **n'envoie rien** sur GitHub. Tant qu'il n'y a pas eu de `git push`, ton travail n'existe que sur ta machine.
- Un `git add` **ne sauvegarde rien**. Il prépare seulement le contenu du prochain commit.
- Un fichier modifié après un `git add` doit être **ré-ajouté** avant le commit, sinon c'est l'ancienne version qui part.

2 Une seule fois, au tout début

Ces étapes ont été faites en semaine 1. Elles ne sont à refaire que sur une **nouvelle machine**, ou pour un **nouveau projet**.

Une fois par machine — ton identité

```
git config --global user.name "Prenom Nom"
git config --global user.email "ton.email@exemple.com"
```

Une fois par projet — créer le dépôt

Le `.gitignore` se crée **avant** le premier `git add` : un fichier entré dans l'historique y reste, même ajouté au `.gitignore` ensuite.

```
cd ~/Documents/Passerelle/Web/miniforum
git init
git branch -M main
git add .
git commit -m "Structure de base du MiniForum"
git remote add origin https://github.com/ton-login/miniforum.git
git push -u origin main
```

Le -u du premier push

Il ne sert qu'une fois : il associe ta branche locale main à celle de GitHub. Ensuite, un simple `git push` suffit pour le reste de l'année.

3 Le rythme d'une séance

En arrivant — récupérer

Première commande de chaque séance, avant d'écrire la moindre ligne :

```
git pull
```

Elle rapatrie ce qui a été poussé depuis une autre machine (celle de la maison, par exemple). Si tu travailles toujours sur le même poste, elle ne fera rien — ce n'est pas une raison pour l'oublier, c'est justement l'oubli qui crée les conflits.

À chaque changement — le cycle de base

C'est le cœur de cette fiche. Dès qu'une chose **fonctionne** ou qu'une étape est **terminée**, tu commites. Pas plus tard, pas « quand tout sera fini ».

```
git status          # qu'ai-je change depuis le dernier commit ?
git add index.html  # je choisis ce qui entre dans ce commit
git commit -m "Formulaire de creation de sujet"
```

Puis on recommence : on code, on teste, on committe. Trois à six commits par séance de TP est un rythme normal.

Qu'est-ce qu'un « changement » qui mérite un commit ?

Un ensemble cohérent, qui laisse le projet dans un état qui **fonctionne** et qui se résume en une phrase. Par exemple :

- « le <header> et la navigation sont en place » → un commit
- « la liste des sujets s'affiche depuis le tableau JavaScript » → un commit
- « j'ai corrigé la faute de frappe dans le titre » → un commit

À l'inverse, « le TP de la semaine 4 » n'est pas un changement : c'est une séance entière, donc plusieurs commits.

git add : nommer ses fichiers ou tout prendre ?

Commande	Quand l'utiliser
<code>git add index.html</code>	Le cas normal. Tu sais exactement ce que tu commites.
<code>git add index.html style.css</code>	Plusieurs fichiers d'un même changement.
<code>git add .</code>	Tout ce qui a changé. Pratique, mais seulement après avoir lu <code>git status</code> .

En partant — envoyer

```
git push
```

Puis rafraîchis la page de ton dépôt sur `github.com`. Si tes fichiers y sont et que l'onglet *Commits* montre bien ceux de la séance, ton travail est en sécurité.

Ne quitte jamais la salle sans avoir poussé

Un commit non poussé n'existe que sur le disque de la machine que tu es en train d'éteindre. Le `git push` de fin de séance est ce qui te permet de reprendre chez toi — et ce qui rend ton travail visible.

4 Écrire un message de commit

Un bon message dit **ce que fait** le commit, pas ce que tu as touché. Il se lit comme une entrée de journal, à l'infinitif ou au participe, court, sans point final.

À écrire	À éviter
Charpente sémantique de la page d'accueil	<code>modifs</code>
Ajout du formulaire de réponse	<code>index.html</code>
Correction du lien vers <code>sujet.html</code>	<code>ça marche</code>
Passage de la liste de sujets en Flexbox	<code>TP semaine 4</code>

Test rapide

Relis ton message dans six semaines. S'il ne te dit pas ce que contenait le commit, il est trop vague. « `modifs` » ne passe jamais ce test.

5 Savoir où j'en suis

Trois commandes qui ne modifient rien et qu'on peut lancer sans risque, aussi souvent qu'on veut.

Commande	Ce qu'elle montre
<code>git status</code>	Les fichiers modifiés, ceux qui sont prêts à être commités, et si tu es en avance ou en retard sur GitHub.
<code>git log --oneline</code>	L'historique, un commit par ligne. Quitter avec la touche <code>q</code> .
<code>git diff</code>	Ligne par ligne, ce que tu as changé depuis le dernier commit. Quitter avec <code>q</code> .

`git status` se lit, il ne se subit pas

La sortie de `git status` contient presque toujours la commande à taper ensuite : Git suggère lui-même `git add`, `git restore` ou `git push` selon la situation. Prends le réflexe de la lire en entier avant de chercher ailleurs.

6 Dépannage

« J'ai oublié de committer pendant deux heures »

Ce n'est pas rattrapable : le passé n'a pas été enregistré. Commite maintenant, en séparant si possible en deux ou trois commits (`git add` fichier par fichier), et reprends le rythme. Un historique fait d'un commit géant par séance est signalé à l'évaluation.

Mon push est refusé (*rejected*)

Le dépôt distant contient des commits que tu n'as pas en local — typiquement, tu as poussé depuis une autre machine. La réponse est presque toujours la même :

```
git pull
git push
```

Git annonce un conflit

Cela arrive quand la même ligne a été modifiée des deux côtés. Git place les deux versions dans le fichier, entourées de marqueurs :

```
<<<<<< HEAD
  <h1>Bienvenue sur MiniForum</h1>
=====
  <h1>MiniForum</h1>
>>>>>> origin/main
```

1. Ouvrir le fichier concerné dans VS Code (il est signalé en couleur dans l'explorateur).
2. Garder la version que tu veux, supprimer l'autre **et les trois lignes de marqueurs**.
3. Enregistrer, puis :

```
git add le-fichier-concerne.html
git commit -m "Résolution du conflit sur le titre"
git push
```

VS Code fait le travail à ta place

Au-dessus de chaque conflit, VS Code affiche les boutons *Accept Current Change* / *Accept Incoming Change* / *Accept Both Changes*. Un clic supprime les marqueurs proprement. Vérifie quand même le résultat avant de committer.

J'ai modifié un fichier et je veux revenir en arrière

Situation	Commande
Annuler mes modifications non committées	<code>git restore fichier.html</code>
Retirer un fichier de l'index (avant commit)	<code>git restore --staged fichier.html</code>
Corriger le message du dernier commit (non poussé)	<code>git commit --amend -m "Nouveau message"</code>

`git restore` détruit du travail

Cette commande écrase tes modifications par la dernière version committée, sans confirmation et sans corbeille. Ne l'utilise que si tu es certain-e de vouloir jeter ce que tu viens d'écrire.

J'ai committé un fichier qui n'aurait pas dû l'être

Cas classique : `node_modules/` ajouté avant que le `.gitignore` soit en place. On le sort du suivi tout en le gardant sur le disque :

```
git rm -r --cached node_modules
git commit -m "Retrait de node_modules du suivi Git"
git push
```

Vérifie ensuite que `node_modules/` figure bien dans le `.gitignore`, sinon il reviendra au prochain `git add`.

Je travaille sur deux machines (école et maison)

La règle tient en une ligne : `git pull` **en arrivant**, `git push` **en partant**, des deux côtés. Si tu oublies le `push` d'un côté, tu retrouveras l'autre machine en retard et tu risques un conflit.

Google Drive n'est pas Git

Le dossier Passerelle est synchronisé sur Drive : c'est une sauvegarde contre la panne de disque. Ce n'est **pas** le canal de transfert entre deux machines. Pour passer d'un poste à l'autre, on passe par GitHub. Laisser Drive et Git écrire en même temps sur le même dossier depuis deux ordinateurs est le meilleur moyen d'abîmer l'historique.

Je suis perdu-e et je veux repartir propre

Un dépôt de référence est publié à la fin des semaines 7, 11 et 17. Pour repartir de celui-ci sans perdre ton travail actuel :

```
cd ~/Documents/Passerelle/Web
git clone https://github.com/.../miniforum-reference.git miniforum-s7
```

Ton ancien dossier reste intact à côté. Aucune pénalité n'est appliquée : c'est prévu par le cours.

Ne supprime jamais le dossier `.git`

Il est caché à la racine de `miniforum` et contient **tout** l'historique du projet. Le supprimer efface l'intégralité de tes commits, définitivement. Aucun problème rencontré dans ce cours ne se règle en effaçant `.git` — viens me voir à la place.

7 Mémo

Commande	Effet
<code>git pull</code>	Récupérer ce qui est sur GitHub. En arrivant.
<code>git status</code>	Faire le point. Aussi souvent que nécessaire.
<code>git add fichier</code>	Préparer un fichier pour le prochain commit.
<code>git commit -m "..."</code>	Enregistrer un point d'étape en local.
<code>git push</code>	Envoyer les commits sur GitHub. En partant.
<code>git log --oneline</code>	Relire l'historique.
<code>git diff</code>	Voir ce qui a changé depuis le dernier commit.
<code>git restore fichier</code>	Jeter les modifications non commitées.

Réflexe de fin de séance

- ☐ `git status` ne signale plus de fichier modifié oublié
- ☐ La séance a produit **plusieurs** commits, aux messages lisibles
- ☐ `git push` a été fait
- ☐ Le dépôt sur `github.com` affiche bien les commits du jour

L'historique compte dans la note

La qualité du code et de l'historique Git représente 10% de la note du projet final, et l'historique est examiné à chaque jalon. Un projet qui apparaît en un seul commit massif donne lieu à un entretien. Committer régulièrement n'est donc pas une coquetterie d'enseignante : c'est la trace de ton travail.